

Informe de hallazgos — Registro de ventas y reporte por categoría

Lenguaje de Programación II · Actividad autónoma, Sesión 9 · Autor: Tineo · Repositorio github.com/Legends600/pharma-frontend, rama `feature/pruebas-reporte-ventas-tineo` · Backend Practica, rama `feature/consultas-reportes` (`e409962`) · Solicitud de incorporación: `feature/pruebas-reporte-ventas-tineo` → `develop` (github.com/Legends600/pharma-frontend/pulls)

La matriz completa está en `docs/matriz-pruebas-s9.md` y las evidencias, nombradas con el ID del caso, en `docs/evidencias-s9/`. Los casos de la SPA se ejecutaron en Chrome; los de la API, con la colección `postman/PharmaSoft-S9-Ventas.json` corrida con Newman (motor de Postman). La parte B agrega el feature `reportes` (`/reportes/ventas-por-categoria`), carga diferida y opción «Reportes» en el menú), verificado en los casos RP-01 a RP-04.

C1. Resumen de la ejecución

Grupo	Casos	Pasan	Fallan	Hallazgo
Buscadores	3 (B-01, B-02, B-03*)	3	0	—
Cálculos	5 (C-01 a C-04, C-05*)	5	0	—
Validaciones	5 (V-01 a V-05)	3	2	H-03 (V-02 por API), H-04 (V-05)
Confirmación	3 (F-01, F-02, F-03*)	1	2	H-01 (F-02), H-02 (F-03)
Mensajes	3 (M-01, M-02, M-03*)	3	0	—
Impresión	1 (I-01)	1	0	—
Consistencia	2 (X-01, X-02*)	2	0	—
Total	22 (17 base + 5 propios*)	18	4	

Todos los cálculos coinciden con la operación hecha a mano y con el total del servidor ($12.50 \times 3 = 37.50$; $18.90 \times 3 = 56.70$; $1.10 \times 3 = 3.30$; $53.60 \rightarrow 15.80$ al quitar la línea del medio). `redondear()` corrige los valores crudos de JavaScript (56.699999999999996 y 3.3000000000000003).

C2. Fichas de hallazgos

H-01 · F-02	Doble clic en «Confirmar venta» registra dos ventas. Se enviaron dos <code>POST /ventas</code> (201 y 201) y se crearon las ventas N.º 28 y 29, idénticas. El botón tiene <code>[disabled]="guardando()"</code> , pero el segundo clic llega antes de que Angular vuelva a pintar el botón deshabilitado.
Severidad	Alta: venta duplicada, stock descontado dos veces y total del día inflado.
Evidencia	<code>F-02_spa.png</code> (registro de red con dos POST 201).
Causa probable	Frontend: <code>VentaForm.confirmar()</code> (<code>venta-form.ts:189</code>) no comprueba si ya hay un registro en curso; depende solo del atributo <code>disabled</code> de la plantilla.
Corrección	SPA (componente) , porque el doble envío nace ahí: <code>if (this.guardando()) return;</code> como primera línea de <code>confirmar()</code> , antes de <code>this.guardando.set(true)</code> . Como defensa adicional en el backend , aceptar una cabecera <code>Idempotency-Key</code> generada al abrir el resumen y rechazar con 409 una segunda venta con la misma clave.
H-02 · F-03	Con el resumen abierto, el formulario sigue editable y «Confirmar» envía datos sin validar. Se cambió la cantidad a 0 después de «Revisar venta» y «Confirmar venta» envió el POST: el backend respondió 400 y la SPA mostró solo «Existen errores de validación», sin indicar la línea.
Severidad	Media: los datos quedan bien (el backend rechaza), pero el resumen ya no corresponde a lo que se confirma y el mensaje no orienta.
Evidencia	<code>F-03_spa.png</code> (POST 400 tras editar con el resumen abierto).
Causa probable	Frontend: <code>revisar()</code> (<code>venta-form.ts:180</code>) valida una sola vez y <code>confirmar()</code> no vuelve a validar; la tabla sigue habilitada mientras <code>confirmando()</code> es true.
Corrección	SPA: en <code>revisar()</code> llamar a <code>this.form.disable()</code> y a <code>this.form.enable()</code> en «Volver» y en el error del POST; además, en <code>confirmar()</code> repetir <code>if (this.form.invalid this.hayExcesoDeStock()) { this.confirmando.set(false); return; }</code> .

H-03 · V-02	La API acepta una cantidad decimal y la trunca. POST /ventas con "cantidad": 2.5 respondió 201 y guardó la venta N.º 26 con cantidad 2 y total 2.20. La SPA sí lo bloquea (<code>Validators.pattern(/^\d+\$/)</code>), así que la regla solo existe en la interfaz.
Severidad	Media: se registra una cantidad distinta de la enviada sin avisar (es la misma causa que el hallazgo H-02 de la sesión 8 en el stock).
Evidencia	V-02b_postman.png (petición con 2.5 → 201 con "cantidad": 2).
Causa probable	Backend: <code>DetalleVentaRequestDTO.cantidad</code> es <code>Integer</code> (línea 29) y Jackson convierte 2.5 en 2 antes de <code>Bean Validation</code> , así que <code>@Min(1)</code> no lo detecta.
Corrección	Backend (deserialización): <code>spring.jackson.deserialization.accept-float-as-int: false</code> en <code>application.yaml</code> y un <code>@ExceptionHandler(HttpMessageNotReadableException.class)</code> en <code>GlobalExceptionHandler</code> que responda 400 con <code>ErrorResponseDTO</code> .

H-04 · V-05	La API acepta el mismo producto en dos líneas. La venta N.º 27 quedó con Loratadina 1 × 12.50 y Loratadina 2 × 25.00. El total (37.50) y el stock (49 → 46) son correctos, pero el comprobante muestra dos líneas del mismo producto y la regla elegida exige una línea por producto (la SPA ya las agrupa en <code>agregarProducto()</code>).
Severidad	Media: datos numéricos correctos, comprobante confuso y detalle distinto del que produce la SPA.
Evidencia	V-05_postman.png (dos líneas con el mismo <code>productoId</code> → 201).
Causa probable	Backend: <code>VentaServiceImpl.registrar()</code> recorre <code>request.getDetalles()</code> (línea 69) sin comprobar repetidos.
Corrección	Backend (servicio), antes del bucle: <pre> Set<Long> vistos = new HashSet<>(); for (DetalleVentaRequestDTO item : request.getDetalles()) { if (!vistos.add(item.getProductoId())) { throw new SolicitudInvalidaException("El producto " + item.getProductoId() + " está repetido en el detalle"); // → 400 } } </pre>

C3. Preguntas de análisis

- ¿Por qué el total de la SPA es solo «estimado»?** Porque se calcula con los precios que la SPA cargó al abrir el formulario (`productoService.listar()` en `VentaForm.ngOnInit()`); si un precio cambia mientras el cajero arma la venta, ese total queda desactualizado. El total definitivo lo calcula `VentaServiceImpl.registrar()` con `producto.getPrecio()` de la base, y por eso `VentaRequest` (`venta.model.ts`) solo lleva `clienteId`, `productoId` y `cantidad`. Si el backend aceptara el total del navegador, bastaría con Postman o las herramientas del navegador para registrar una venta de S/ 100 cobrando S/ 0.01: el precio quedaría en manos del cliente y los reportes de ventas dejarían de ser confiables.
- V-05: ¿aceptar el mismo producto en dos líneas?** No. Con el descuento condicional (`ProductoRepository.descontarStock()`) el stock final queda bien, pero cada línea se valida por separado: con stock 2 y líneas de 2 y 1, la segunda línea pasa la verificación previa (el producto leído todavía dice 2) y luego falla el `UPDATE`, de modo que el cajero recibiría «otra venta acaba de usar las unidades», un mensaje falso. Además, el comprobante muestra el producto dos veces y las cantidades por línea ya no representan «cuántas unidades de X se vendieron». Lo coherente es que la API rechace el detalle con 400 (H-04), igual que la SPA, que agrupa la cantidad en una sola línea.
- Mejora después del 409 de M-01.** Hoy la SPA conserva la venta y muestra el mensaje (el cajero pudo corregir sin recargar, venta N.º 31), pero su lista de productos sigue diciendo «Stock 2». Aplicaría: (a) volver a pedir los productos tras el 409 para actualizar el signal `productos`, con lo que `excedeStock` marcaría la línea al instante; (b) que el backend incluya el `productoId` y el stock disponible en el error, para resaltar esa fila y ofrecer «Ajustar a 0 disponibles / Quitar línea»; y (c) mover el foco a esa línea. Así el cajero corrige en un paso en lugar de buscar el problema.
- ¿Por qué el reporte no se pagina y la consulta sí?** El reporte agrega las ventas con `GROUP BY` por categoría (`VentaRepository.reporteVentasPorCategoria()`): devuelve una fila por categoría, una cantidad pequeña y acotada que no crece con el número de ventas. Además, para la fila de totales (`totalUnidades` y `totalMonto`) y para el ancho de cada barra (`mayorMonto`) la vista necesita todas las filas a la vez. La consulta de ventas, en cambio, devuelve una fila por venta y crece sin límite con el tiempo: sin paginación, cada búsqueda traería miles de ventas con sus detalles.